

Alpha-Beta Pruning

1. Aim

To implement the **Alpha-Beta Pruning algorithm** to efficiently search a game tree and determine the best possible move by eliminating branches that cannot affect the final decision.

2. Problem Statement

In Artificial Intelligence, game-playing systems need to examine different possible moves to select the best move. The **Minimax algorithm** can require searching many nodes in a game tree, which increases the computational cost. Alpha-Beta Pruning improves this process by removing branches that do not need to be explored while producing the same optimal result as Minimax.

3. Solution

The problem is solved using the **Alpha-Beta Pruning technique**, an optimization of the Minimax algorithm. It maintains two values, **alpha** and **beta**, while searching the game tree. Alpha represents the best value that the maximizing player can guarantee, while beta represents the best value that the minimizing player can guarantee. When a branch cannot produce a better result, it is pruned and further exploration of that branch is avoided.

4. Algorithm

1. Start at the root node of the game tree.
2. Set **alpha** = $-\infty$ and **beta** = $+\infty$.
3. If the current node is a terminal node, return its value.
4. For a maximizing node, select the maximum value among its children.
5. Update the alpha value with the best value found.
6. If **alpha** $\geq$ **beta**, stop exploring the remaining children because they cannot affect the result.
7. For a minimizing node, select the minimum value among its children.
8. Update the beta value with the best value found.
9. If **alpha** $\geq$ **beta**, prune the remaining branches.
10. Continue until the root node is evaluated.
11. Return the best possible value or move.

5. Program

```
def alpha_beta(node, depth, alpha, beta, maximizing):  
    # Leaf node  
    if isinstance(node, int):  
        return node
```

```
if maximizing:
```

```
    best = float('-inf')
```

```
    for child in node:
```

```
        value = alpha_beta(  
            child, depth + 1, alpha, beta, False  
        )
```

```
        best = max(best, value)
```

```
        alpha = max(alpha, best)
```

```
        # Alpha-Beta pruning
```

```
        if alpha >= beta:
```

```
            break
```

```
    return best
```

```
else:
```

```
    best = float('inf')
```

```
    for child in node:
```

```
value = alpha_beta(  
    child, depth + 1, alpha, beta, True  
)
```

```
best = min(best, value)
```

```
beta = min(beta, best)
```

```
# Alpha-Beta pruning
```

```
if alpha >= beta:
```

```
    break
```

```
return best
```

```
# Game tree
```

```
tree = [  
    [  
        [3, 5],  
        [6, 9]  
    ],  
    [  
        [
```

```
[1, 2],  
[0, -1]  
]  
]  
  
result = alpha_beta(  
    tree,  
    0,  
    float('-inf'),  
    float('inf'),  
    True  
)
```

```
print("Optimal Value:", result)
```

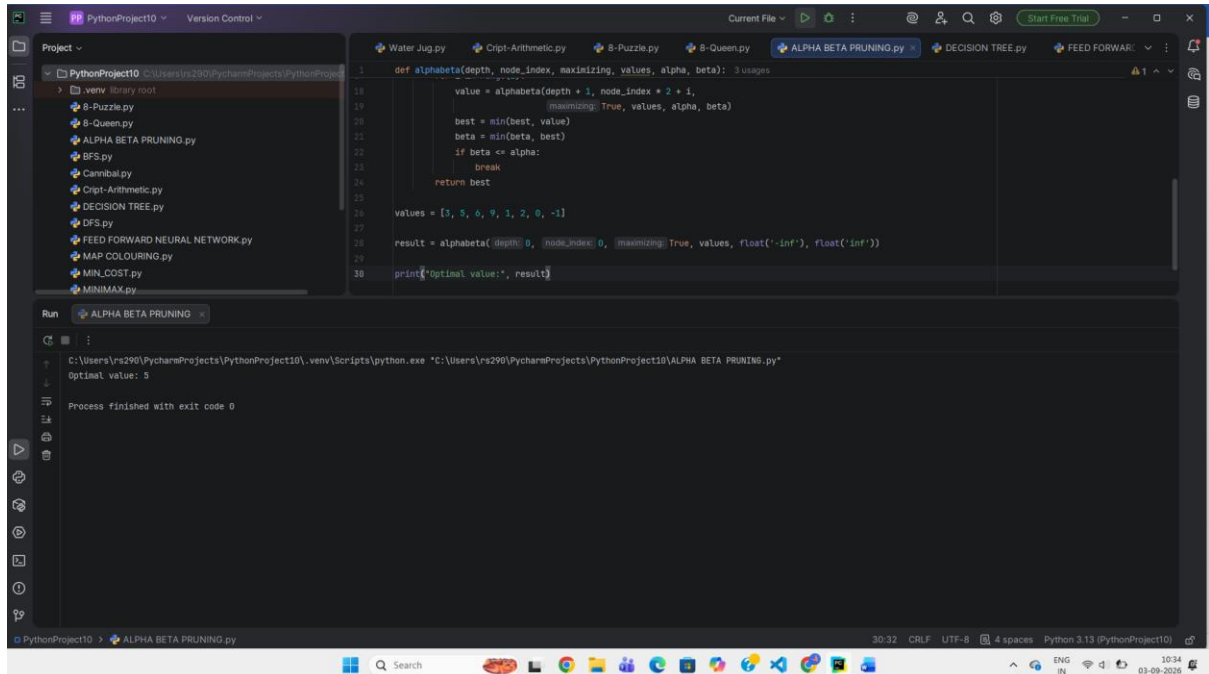
6. Result

The Alpha-Beta Pruning algorithm was successfully implemented in Python. The algorithm evaluated the required nodes of the game tree and pruned branches that could not influence the final decision.

Output:

Optimal Value: 3

Thus, the **Alpha-Beta Pruning** algorithm successfully determined the optimal value while reducing unnecessary exploration of the game tree.



The screenshot displays the PyCharm IDE interface. The left sidebar shows a project named 'PythonProject10' with a file explorer containing various Python files, including 'ALPHA BETA PRUNING.py'. The main editor window shows the code for the 'alpha_beta_pruning' function. The function is defined as follows:

```
def alpha_beta(depth, node_index, maximizing, values, alpha, beta):
    value = alpha_beta(depth + 1, node_index + 1,
                        maximizing, values, alpha, beta)
    best = min(best, value)
    beta = min(beta, best)
    if beta <= alpha:
        break
    return best

values = [3, 5, 6, 9, 1, 2, 0, -1]
result = alpha_beta(0, 0, maximizing=True, values, float('-inf'), float('inf'))
print('Optimal value:', result)
```

The bottom panel shows the 'Run' output for 'ALPHA BETA PRUNING'. The output indicates that the process finished with exit code 0 and the optimal value is 5.

```
Run ALPHA BETA PRUNING
C:\Users\rs290\PycharmProjects\PythonProject10\venv\Scripts\python.exe "C:\Users\rs290\PycharmProjects\PythonProject10\ALPHA BETA PRUNING.py"
Optimal value: 5
Process finished with exit code 0
```